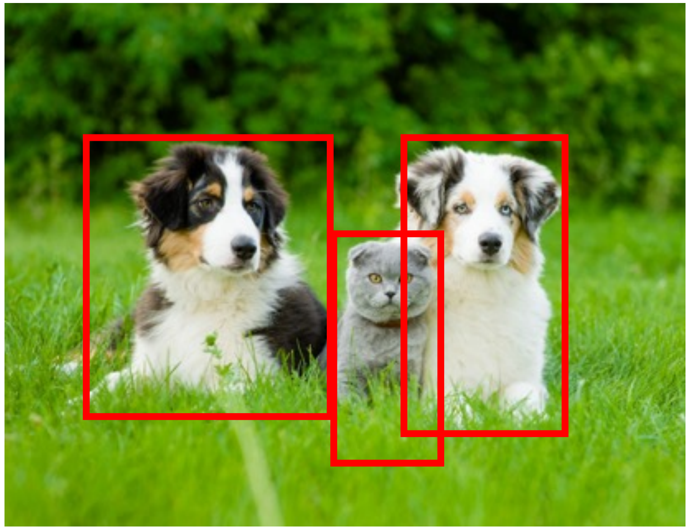
Finding and Classifying Objects



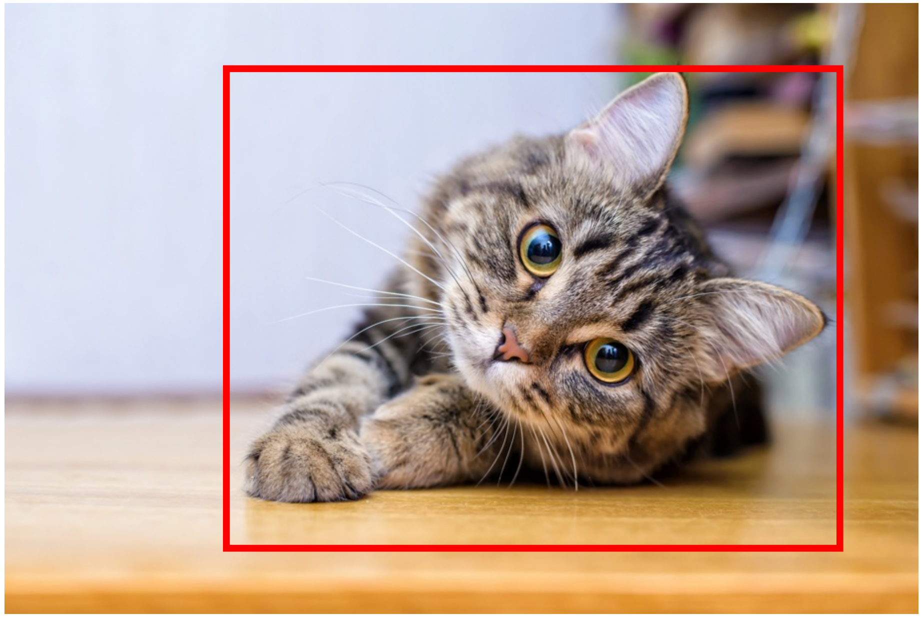
Computer Science

Prof. Dr. Thomas Koller

Problems in Computer Vision

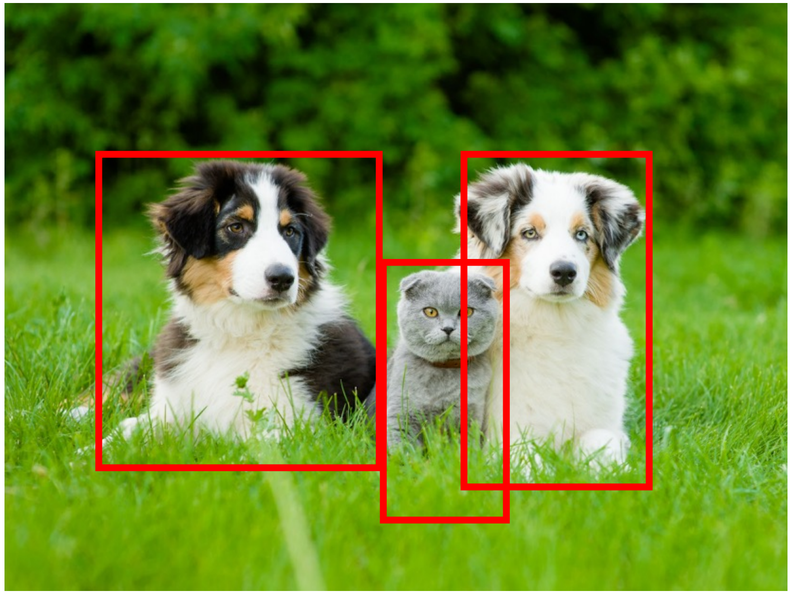
Image Classification

Classification and Localization



Problems in Computer Vision

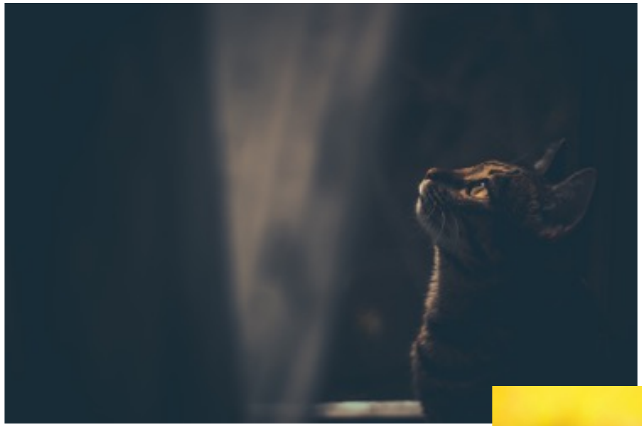
Object Detection and Classification

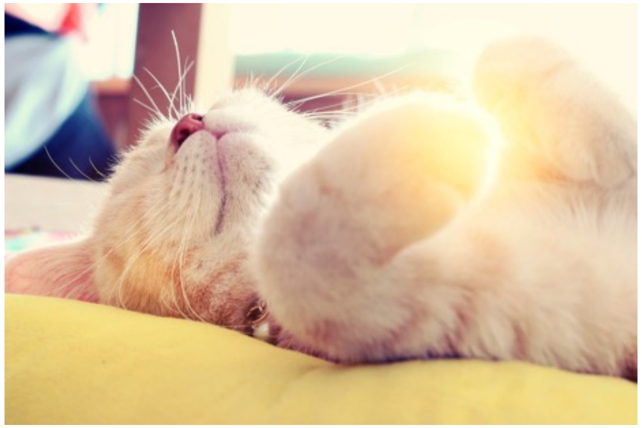


Semantic Segmentation

Challenges: Different Backgrounds / Clutter

Challenges: Illumination





Challenges: Occlusion

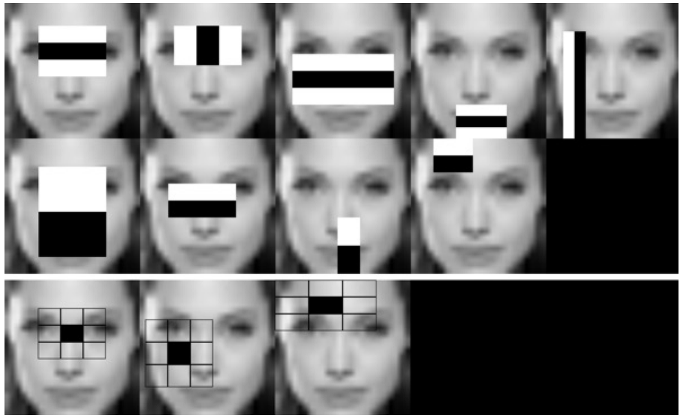
Challenges: Deformation

Challenges: Interclass Variations

Object Detection & Classification

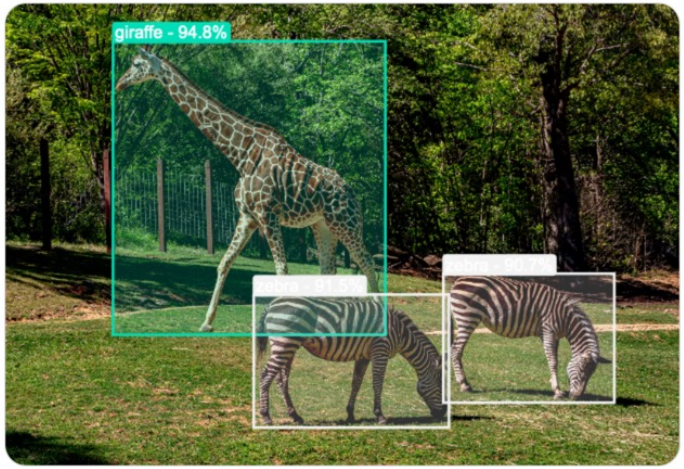
Classic Algorithms (today):

- Cascaded Classifier
- Viola-Jones Face Detector



Modern Algorithms (next lecture):

- RCNN
- Fast and Faster RCNN
- YOLO and SSD



Face Detection

Nikon Advertisement

Viola Jones Face-Detektor

Problem:

- Sliding Window: Evalulate many position in the image if they contain a face
- How to scan many positions as efficiently as possible for possible faces?
- How to find good features?

Ideas:

- ***Boosting*** using Adaboost
- **Simple** Features using Haar Wavelets
- ***Cascaded Classifiers***

Robust Real-time Object Detection

Paul Viola
viola@merl.com
Mitsubishi Electric Research Labs
201 Broadway, 8th FL
Cambridge, MA 02139

Michael Jones
mjones@crl.dec.com
Compaq CRL
One Cambridge Center
Cambridge, MA 02142

Machine Learning Approach: Boosting

Idea:

Input: Training data (positive, negative) with **weights**

FOR t=1...T
- Train a *weak* classifier
- Classify training data
- Increase weight on incorrectly classified samples

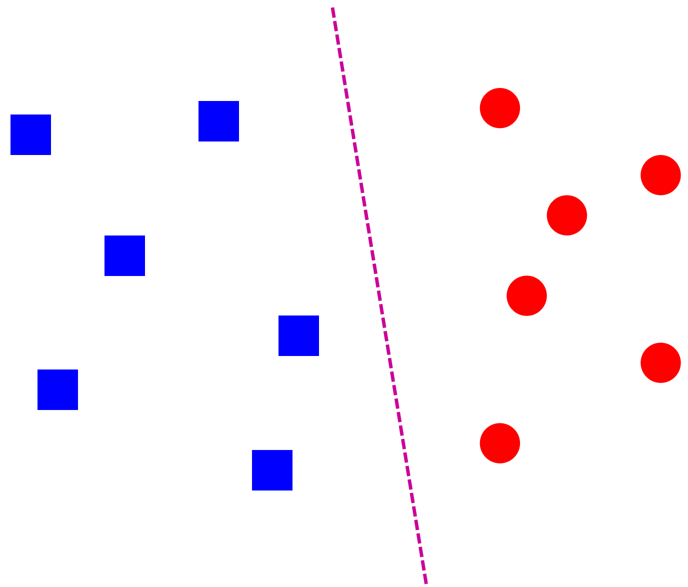
Use the weighted combination of all weak classifiers for the final (strong) classifier

Classification: Linear function

Use a linear function (also called hyperplane) to distinguish between 2 classes

$$f(\boldsymbol{x}) = \boldsymbol{w}\boldsymbol{x} + \boldsymbol{b}$$

Output is positive or negative depending on which side a sample is



Boosting

- *Strong* classifier from a linear combination of *weak* classifiers

weights

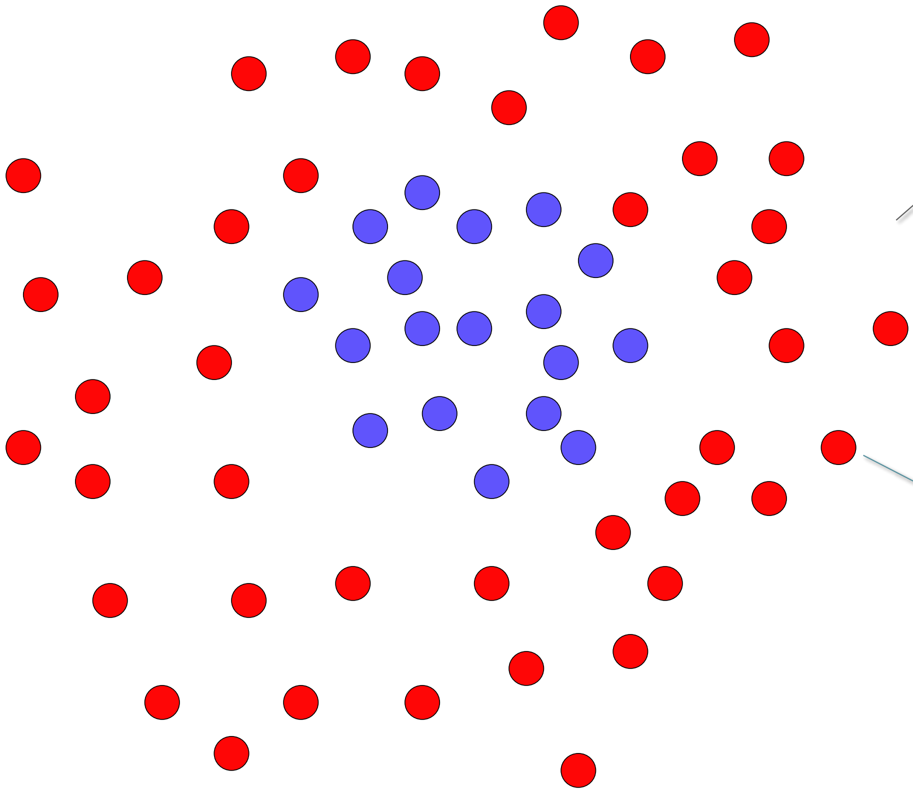
$$F(x) = \alpha_1 f_1(x) + \alpha_2 f_2(x) + \alpha_3 f_3(x) + \cdots$$

strong classifier

weak classifiers

The weak classifiers are **linear functions**

Boosting

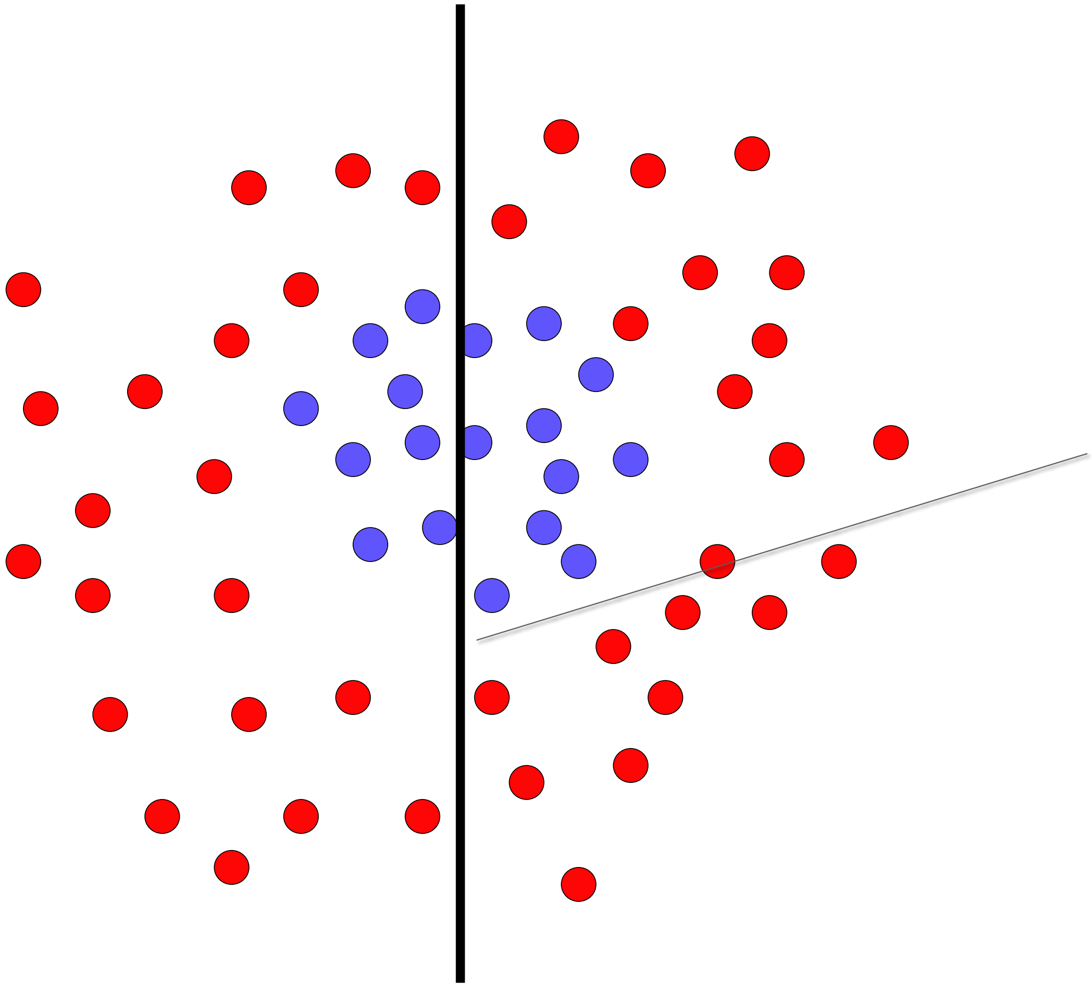


Goal:

Find a classifier that separates the red from the blue samples

Each data point has a class label (+1, -1) and a weight (initialized to 1)

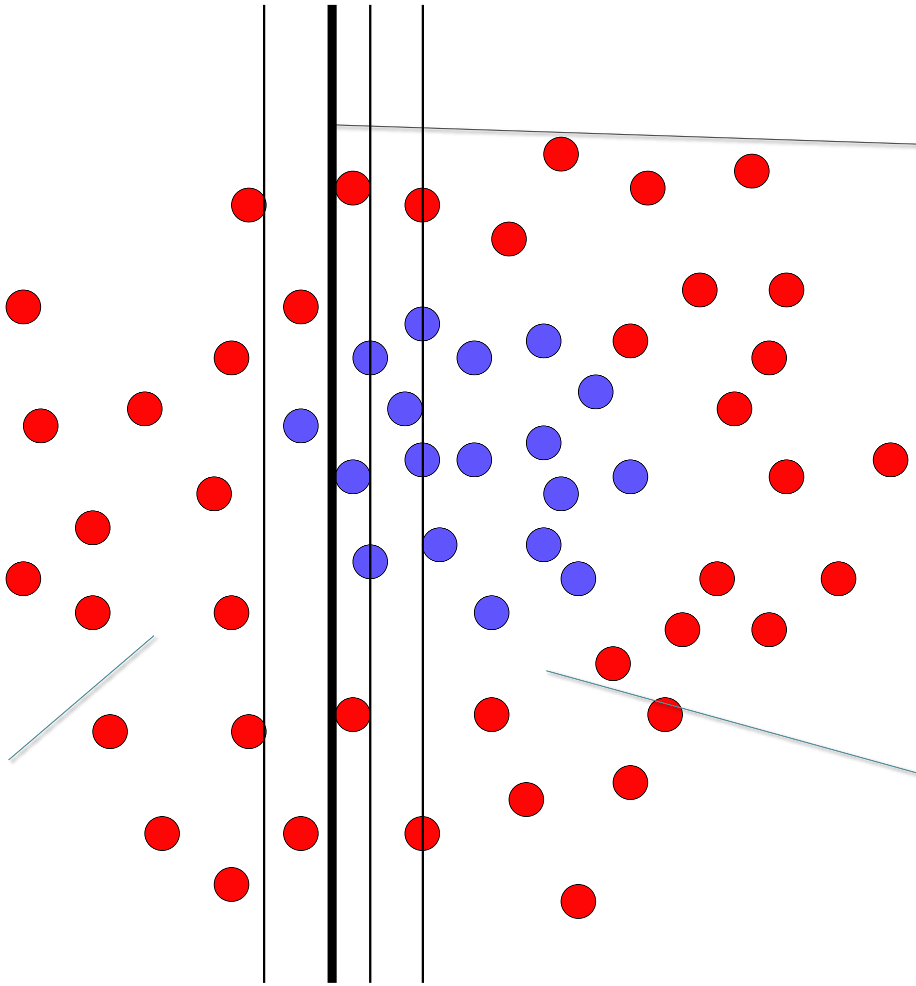
Boosting



weak linear classifier

p(error) = 0.5

Boosting



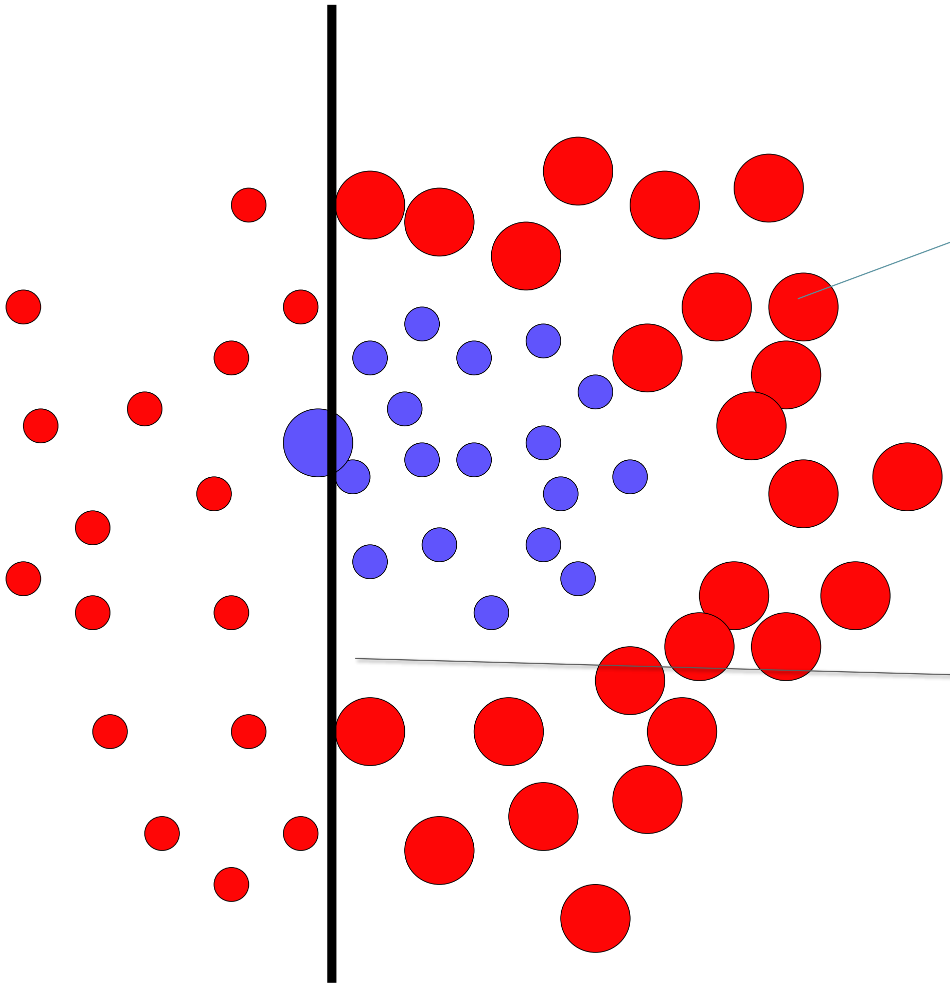
This one seems best

It is still a **weak classifier**: it performs slightly better than chance

This side will be classified as red

This side will be classified as blue

Boosting

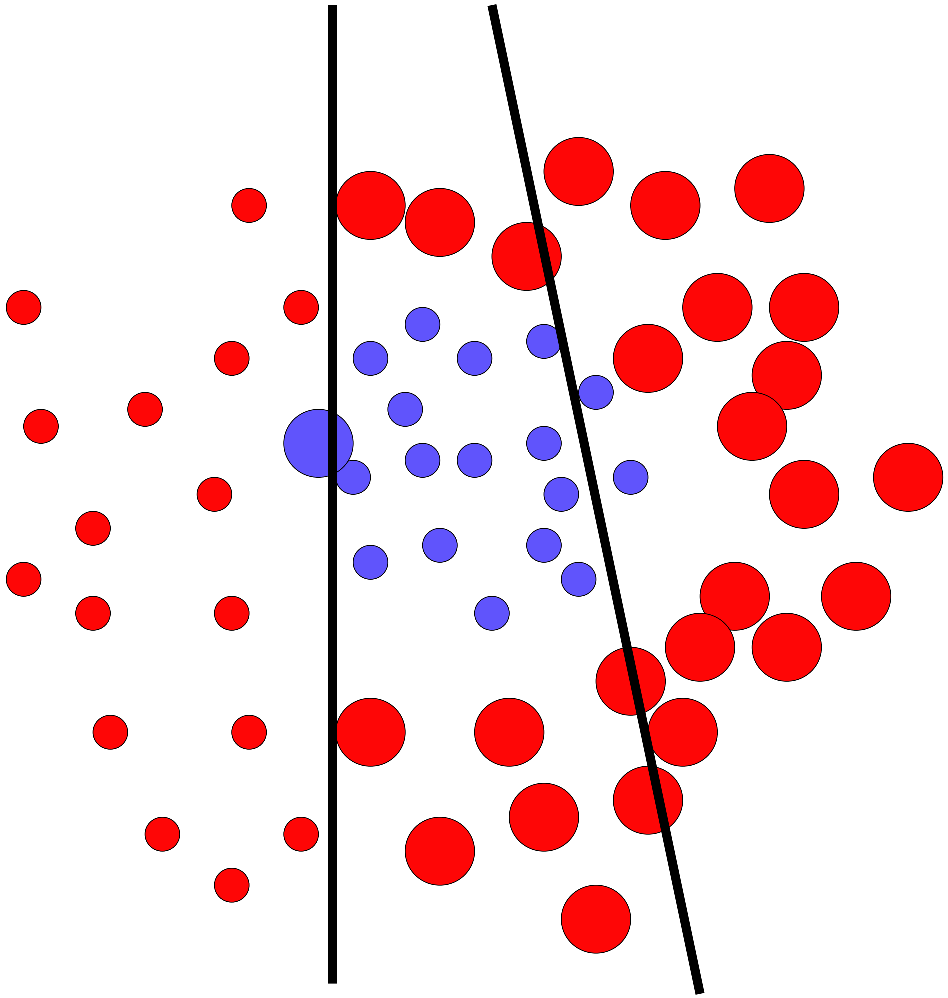


We update the weights:

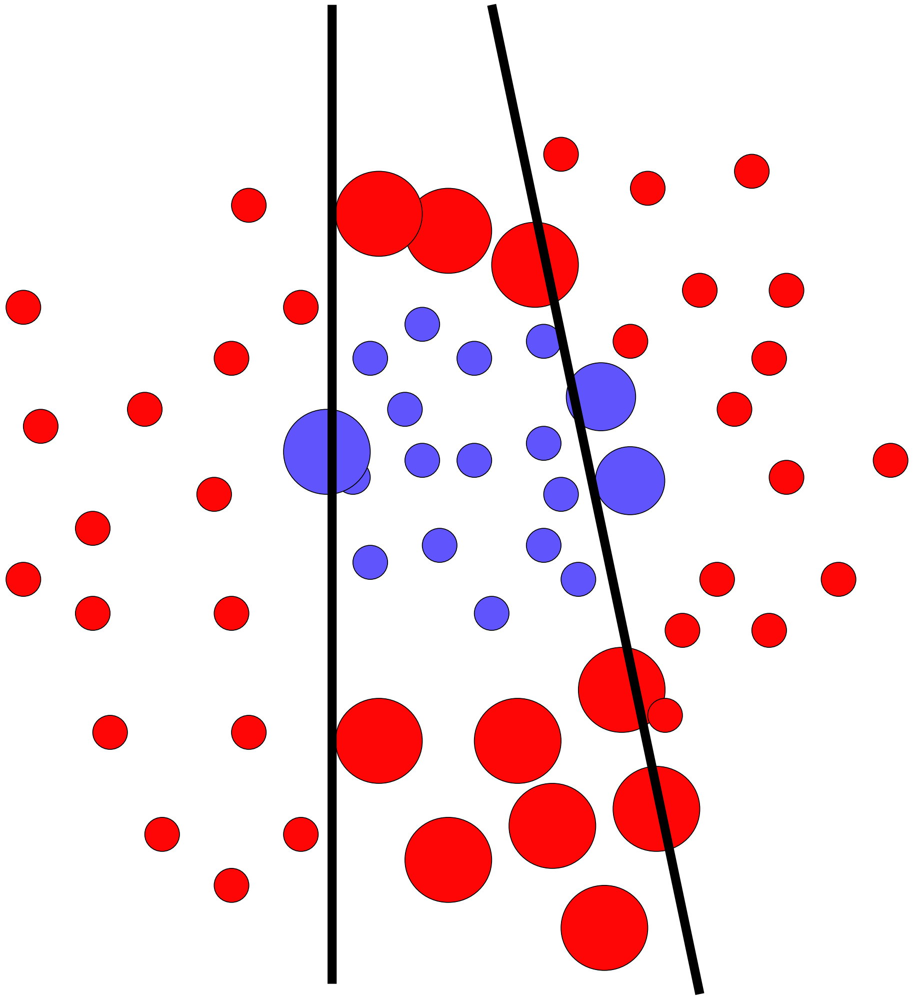
- Increase the weights for wrongly classified samples
- Decrease the weights for correctly classified samples

This sets a new problem for which the previous weak classifier performs at chance again

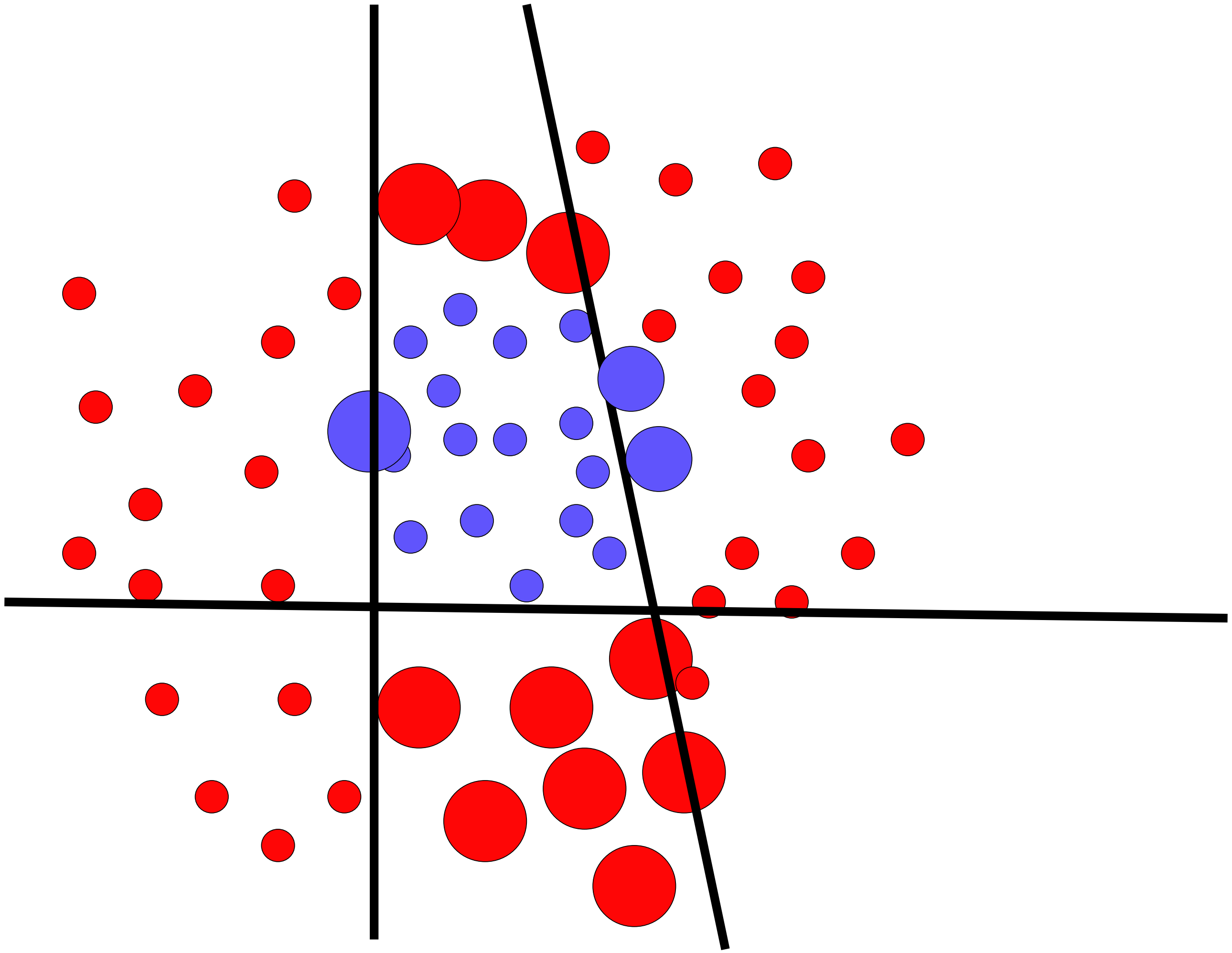
Boosting



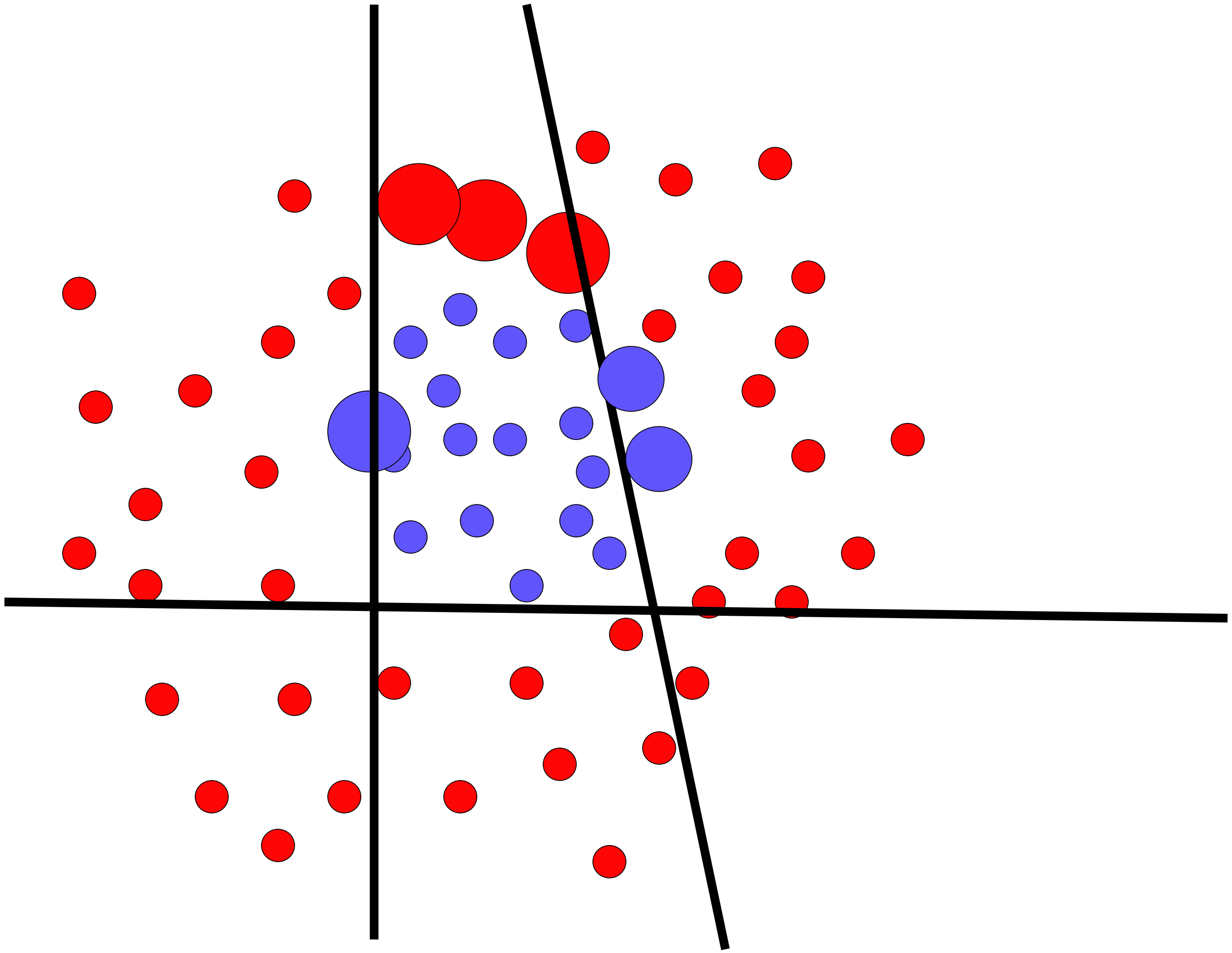
Boosting



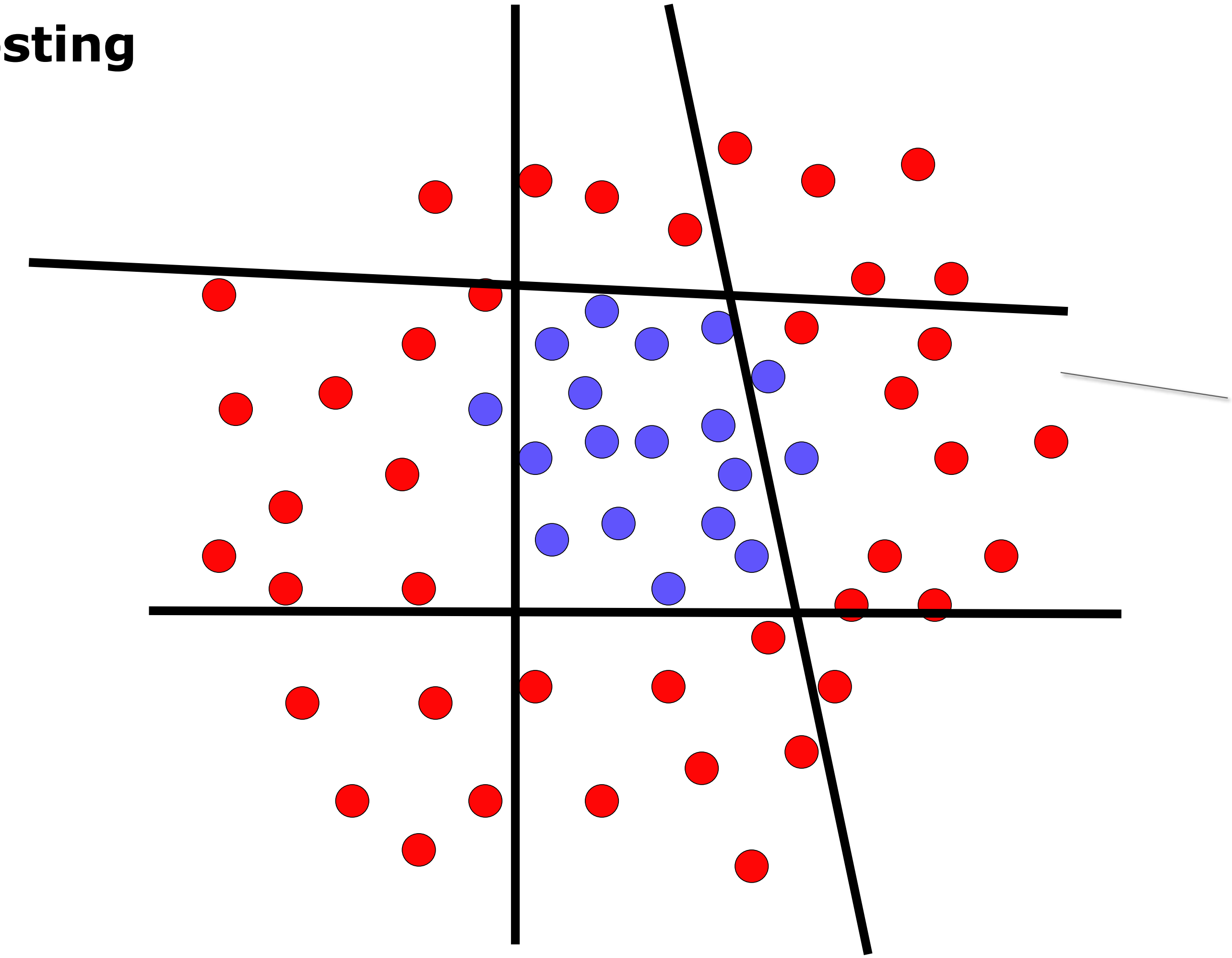
Boosting



Boosting



Boosting



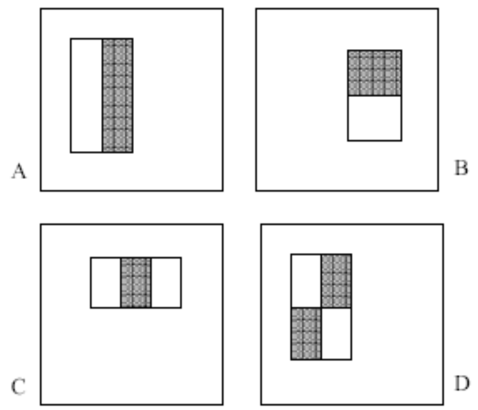
The final strong (non-linear) classifier is built as the combination of all the weak (linear) classifiers

Features

Filter responses (convolutions) can be used as features but are computationally expensive

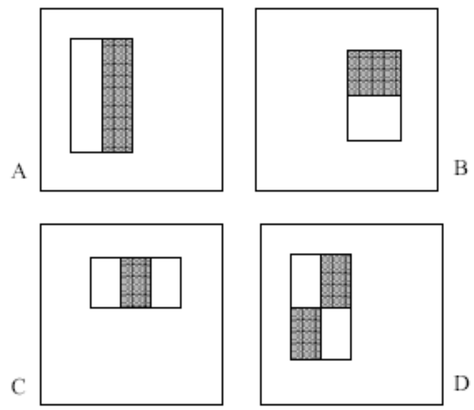
Idea:

- Use simple filters of different shapes and sizes with only -1 or +1 in the filter masks
- These are called *Haar Features or Haar Wavelets*



Feature Extraction

- Calculate features on sub windows (sliding window)
- Sub-Image 24x24 Pixel
- All possible combination of filters are feature candidates: 160‘000 combinations



Integral image

- Fast calculation of filter results by using integral images
- Each value in the integral image contains the sum of all pixel value in the upper, left rectangle also called summed area tables...

Integral Image

Original Image

0	1	3	5	2	7	10	7	10	9
3	7	6	9	3	8	1	8	5	8
1	11	0	7	13	2	14	2	13	1
14	3	2	7	1	0	9	7	2	12
1	5	15	3	6	6	5	1	10	6
8	1	2	6	7	3	2	11	0	15
7	7	6	0	9	5	10	3	8	1
12	5	6	10	11	3	6	7	9	1

Original Image

0	1	3	5	2	7	10	7	10	9
3	7	6	9	3	8	1	8	5	8
1	11	0	7	13	2	14	2	13	1
14	3	2	7	1	0	9	7	2	12
1	5	15	3	6	6	5	1	10	6
8	1	2	6	7	3	2	11	0	15
7	7	6	0	9	5	10	3	8	1
12	5	6	10	11	3	6	7	9	1

Integral Image

		51							

https://levelup.gitconnected.com/the-integral-image-4df3df5dce35

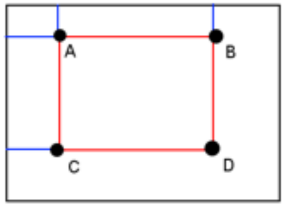
Integral Image

Original Image

0	1	3	5	2	7	10	7	10	9
3	7	6	9	3	8	1	8	5	8
1	11	0	7	13	2	14	2	13	1
14	3	2	7	1	0	9	7	2	12
1	5	15	3	6	6	5	1	10	6
8	1	2	6	7	3	2	11	0	15
7	7	6	0	9	5	10	3	8	1
12	5	6	10	11	3	6	7	9	1

Integral Image

0	1	4	9	11	18	28	35	45	54
3	11	20	34	39	54	65	80	95	112
4	23	32	53	71	88	113	130	158	176
18	40	51	79	98	115	149	173	203	233
19	46	72	103	128	151	190	215	255	291
27	55	83	120	152	178	219	255	295	346
34	69	103	140	181	212	263	302	350	402
46	86	126	173	225	259	316	362	419	472



Sum = D - B - C + A

Sum = D - B - C + A = 215 - 72 - 80 + 20 = 83

Classification

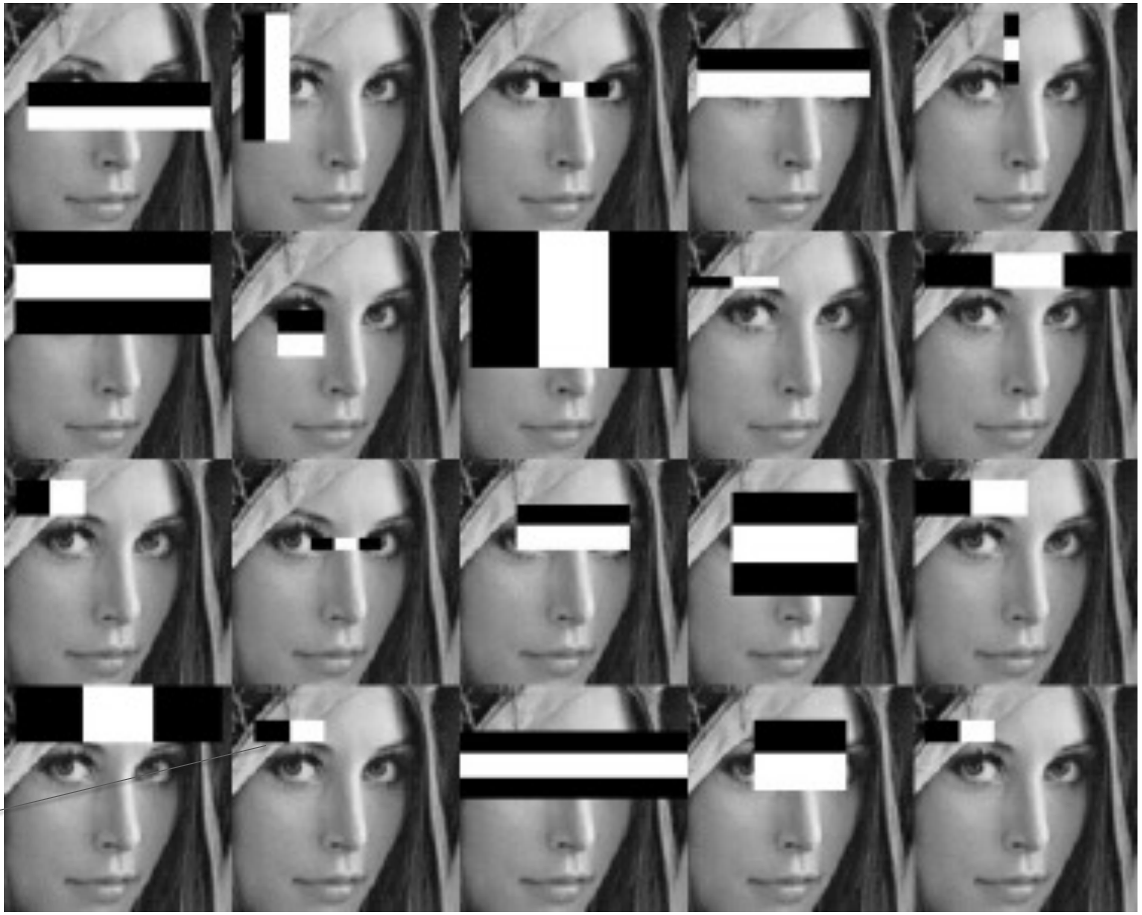
Problem:

- How to evaluate 160‘000 features per sub window?

Idea:

- Treat features as weak classifiers
- Train important features using adaboost

Each filter response is a classifier for a face

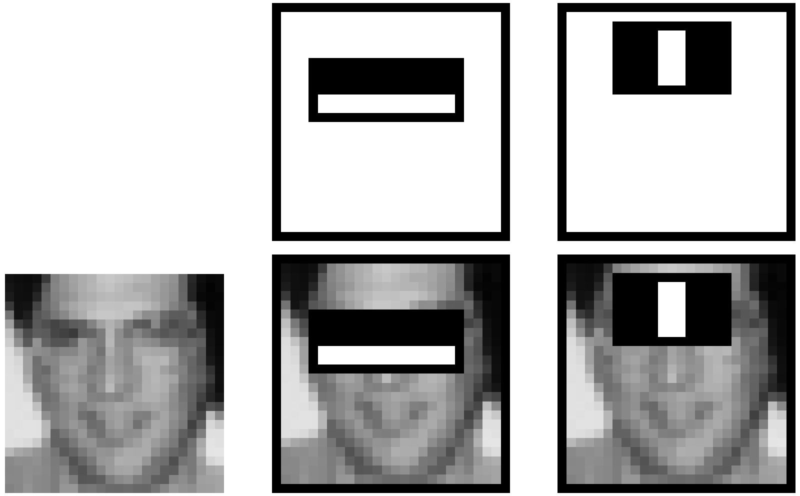


Feature selection by Adaboost

The first and second features selected by Adaboost.

The first feature measures the difference in intensity between the eye region and the face below.

The second feature compares the intensities in the eye region to the intensity across the bridge of the nose.



Efficient Face Detection

A sliding window detector needs to evaluate thousand of possible position/scae combinations in the image

Faces are rare: 10-12 faces per image

Using 1'000'000 pixels should result in a false positive rate of less than 10^{-6}

Idea:

- Dismiss regions without faces efficiently

Goal for face detection:

- high true positive rate (85-95%)
- Low false positive rate (10^{-5} bis 10^{-6})

Cascade approach:

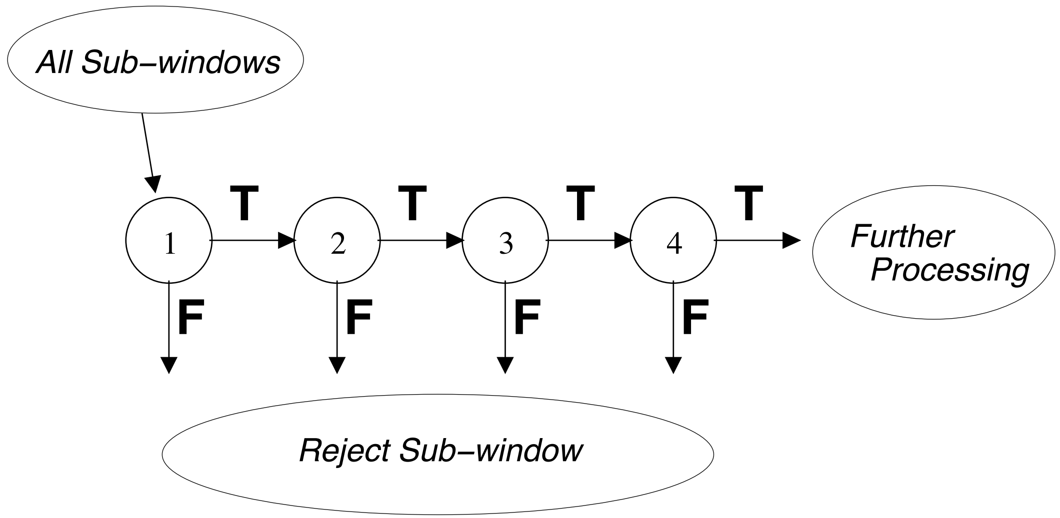
- Use a sequence of individual classifiers:
- False positive rates multiply
- True positive rates multiply

Cascaded Classifiers Approach

Reject false results early in the processing.

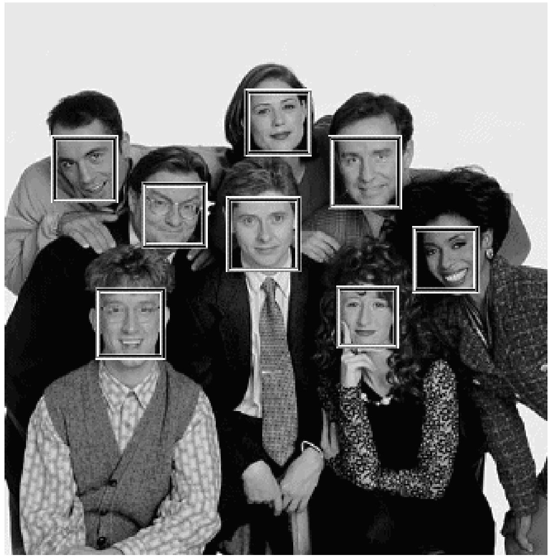
With the overall goal, we can calculate the necessary goal for each step of the cascade:

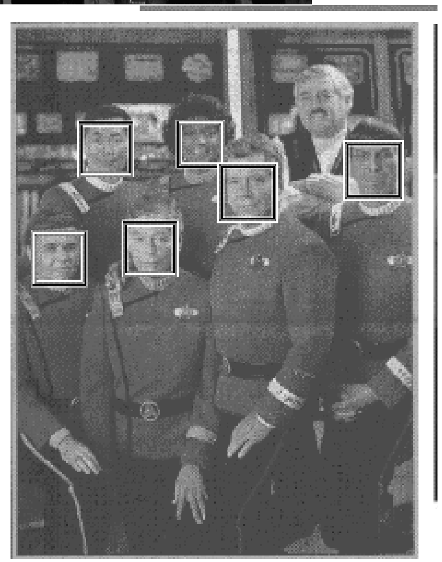
- 0.99 True positive rate
- 0.3 False positive rate



Each step of the 10-step cascade is trained with adaboost using 20 (different) features

Training data





Results

Example: 22 stage cascade

MASTER OF SCIENCE
IN ENGINEERING

Thank You!

School of Computer Science and Information Technology
Degree Programs
Prof. Dr. Thomas Koller
Head of the Master's Program in Engineering

Phone direct +41 41 757 68 32
thomas.koller@hslu.ch